메모리 분석 우회 기법 연구 및 개선방안 도출

22조 MalGround

INDEX

1. 팀 소개

01

디지털 포렌식
김종민 멘토님

디지털 포렌식
김종현 멘토님

디지털 포렌식
PM 이한얼

디지털 포렌식
김태우

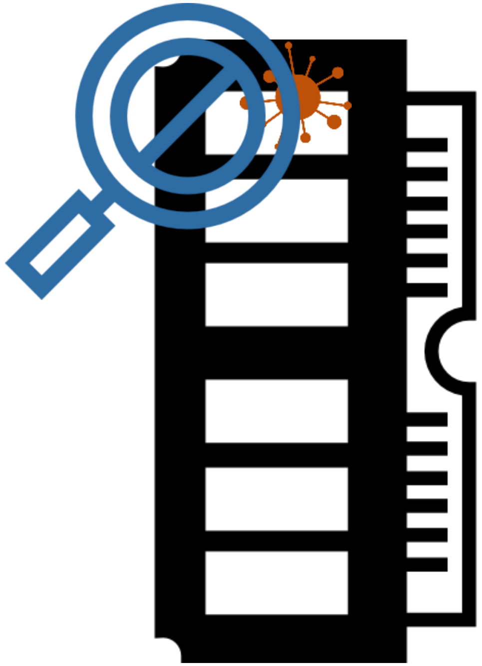
디지털 포렌식
전은진

보안 제품 개발
김한솔

보안 제품 개발
한지혁

02

메모리 포렌식



- 모든 실행된 악성코드는 메모리를 거침
- 특정 시점의 메모리를 이미징 하여 시스템과 독립적으로 흔적 추출
- 사고대응, 악성코드 분석 등에 활용

02

메모리 분석 우회 기법 연구 및 개선방안 도출

침해 사고 발생 시

메모리 포렌식 툴을 사용 하여 분석

이를 우회하는 방안을 연구하고, 대응책을 마련

02

[2004]
Windows 커널 공격 기법 대응 모델 및 메커니즘 연구
윈도우 루트킷 탐지, 차단 기법에 관한 연구
[2008]
윈도우 환경에서 효과적인 루트킷 탐지에 대한 연구
[2010]
윈도우 커널 기반 API 후킹 탐지 기법
윈도우 물리 메모리의 커널 객체 분석 방안 및 도구 구현

~2010

2012

메모리 변조에 의한
은닉 파일 탐지도구
설계 및 구현

Black Hat)
One-byte Modification
for Breaking Memory
Forensic Analysis
Infigo)
Defeating Windows
memory forensics

2013

DFRWS) Anti-Forensic
Resilient Memory Acquisition

2015

필수 호출코드 테이블을 활
용한 힙 메모리 상주형 악성
코드 탐지기법 연구

윈도우 환경에서
프로세서 제어영역을 통한
안티메모리 포렌식 대응기법

2017

PHYSICAL_MEMORY_RUN
메모리 구조체 조작을 이용한
안티 덤프 기법

2018

명령어 가상 실행을 이용한
적시 프로세스 메모리
암호화 연구

02

[2004]
Windows 커널 공격 기법 대응 모델 및 메커니즘 연구
윈도우 루트킷 탐지, 차단 기법에 관한 연구
[2008]
윈도우 환경에서 효과적인 루트킷 탐지에 대한 연구
[2010]
윈도우 커널 기반 API 후킹 탐지 기법
윈도우 물리 메모리의 커널 객체 분석 방안 및 도구 구현

메모리 변조에 의한
은닉 파일 탐지도구
설계 및 구현

코드 탐지기법 연구

~2010 | 2012 | 2013 | 2016 | 2017 | 2018

Black Hat)
One-byte Modification
for Breaking Mem
Forensic Analysis
Infigo)
Defeating Windows
memory forensics

DFRWS) Anti-Forensic
Resilient Memory Acquisition

PHYSICAL_MEMORY_RUN
메모리 구조체 조작을 이용한

안티메모리 포렌식 대응기법

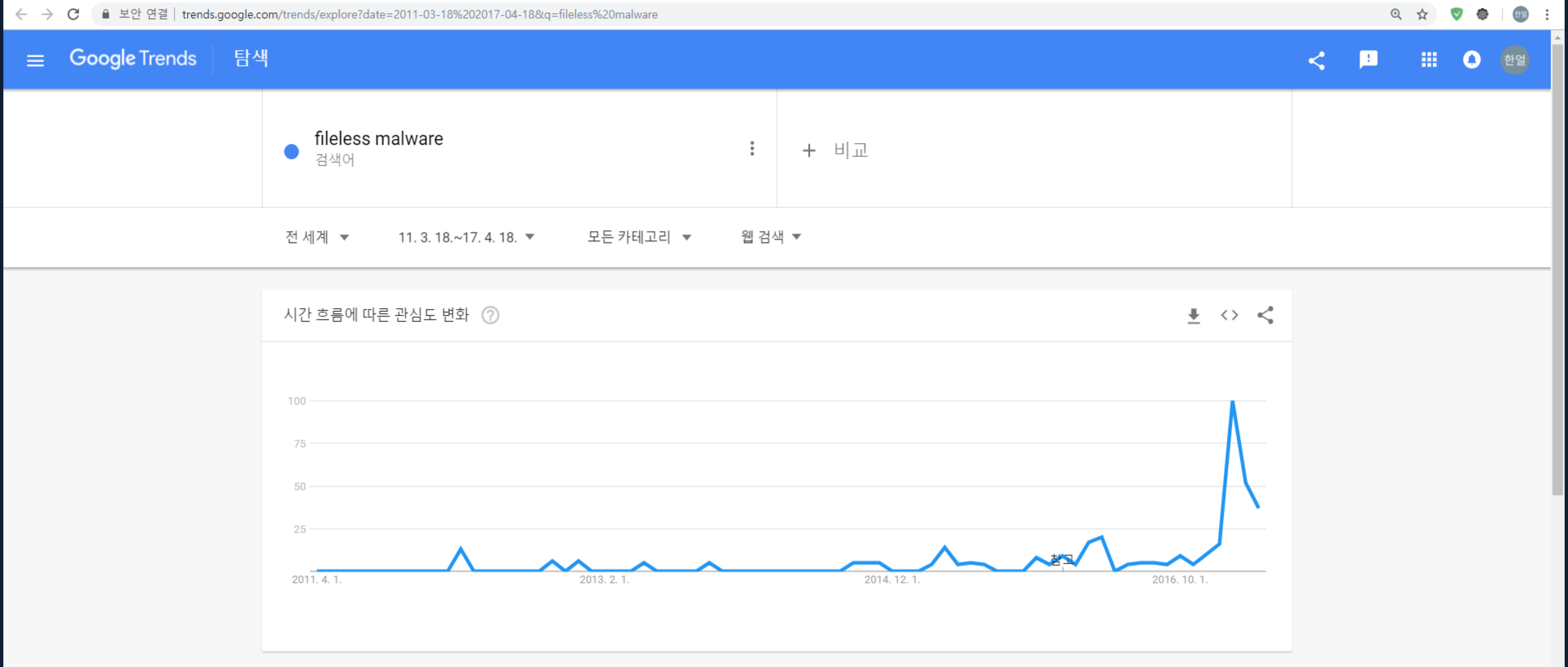
암호화 연구

"악성 실행 전 과정은 **런 타임 메모리에서 발생**한다.
따라서 기존의 탐지에 기반한 파일 스캔 프로그램에
들키지 않고 우회할 수 있다." – McAfee Labs (2018)

" 파일기반 탐지기술로는 파일리스 악성코드를 탐지하기 어렵다.
메모리 기반 악성코드 진단을 강화하거나, PowerShell처럼 취약점 활용에
동원되는 기능을 차단해 피해를 예방해야 한다." - 하우리(2017)

http://www.itworld.co.kr/t/36/%EB%B3%B4%EC%95%88/110264#csidxb2c7be700835562bd6ad086ba462189
http://m.zdnet.co.kr/news_view.asp?article_id=20170116143243#imadnews

Fileless malware 에 대한 관심도_Google Trends



악성코드가 메모리에서 발견되지 않는다면?

3. 우회 및 대응

< 우회 방안>

1) **수집 방해**

2) **분석 방해**

< 대응 방안 >

우회된 악성코드의 흔적을 찾아낼 수 있는 방법 제시

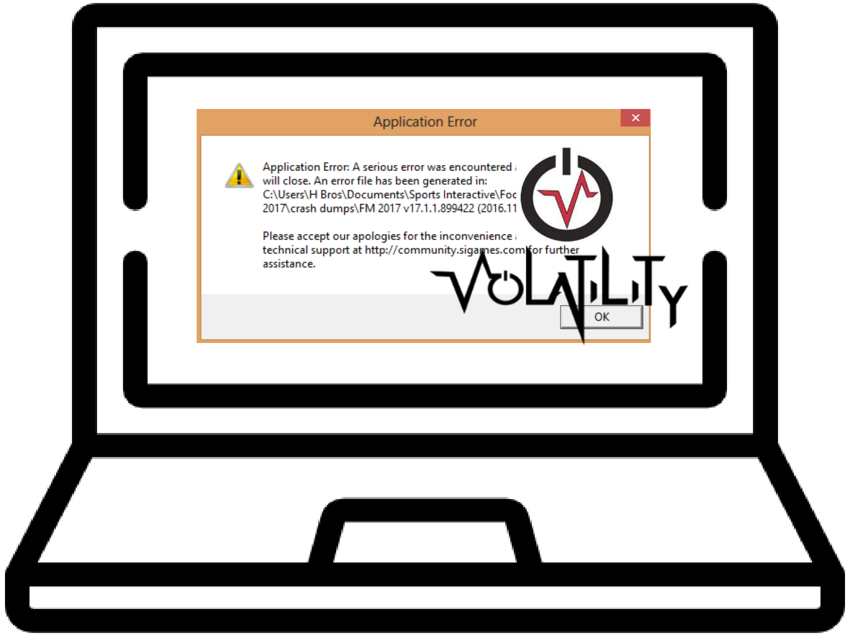
+ 우회된 악성코드를 찾아 내는 플러그인 제작

1) 수집 방해

< 사고 현장 >

- **데이터 수집 단계**에서 수집을 방해하는 방법
  - 수집하는 행위를 방해
  - 수집된 데이터를 조작

2) 분석 방해



< 데이터 수집 후 분석 >

- **데이터 분석 단계** 에서 분석을 어렵게 하는 방법
  - 분석 도구 우회 로직

04

공격 기법

Rootkit

- Hooking(**SSDT**, IDT)
- DKOM

Fileless

- **Powershell**
- Python

Injection

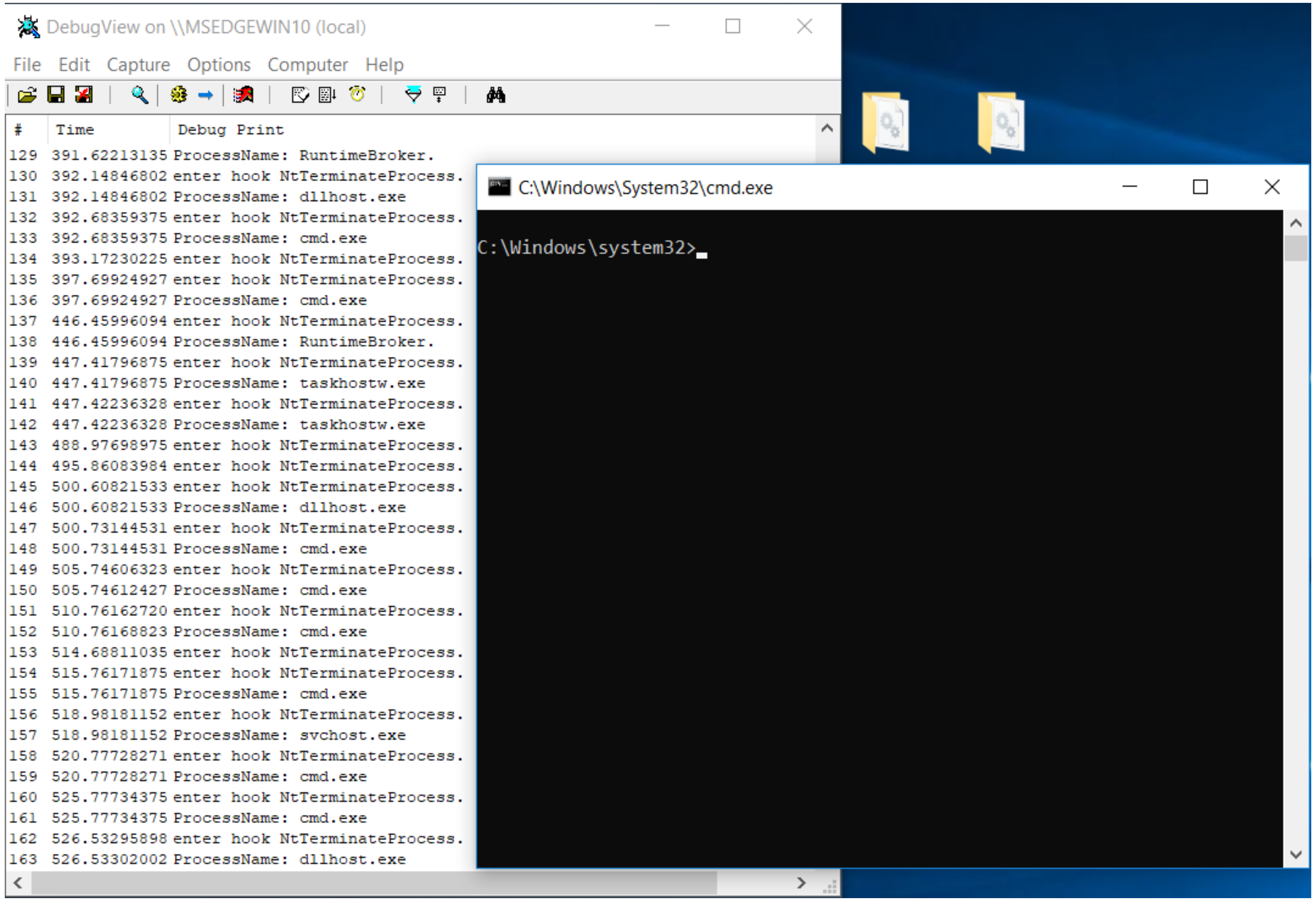
- Code Injection
- DLL Injection

?

04

Rootkit | Fileless | Injection

SSDT Hooking



1) SSDT hooking이 적용된 드라이버 로드

2) 메모리상의 드라이버 흔적 확인

04

Rootkit | Fileless | Injection

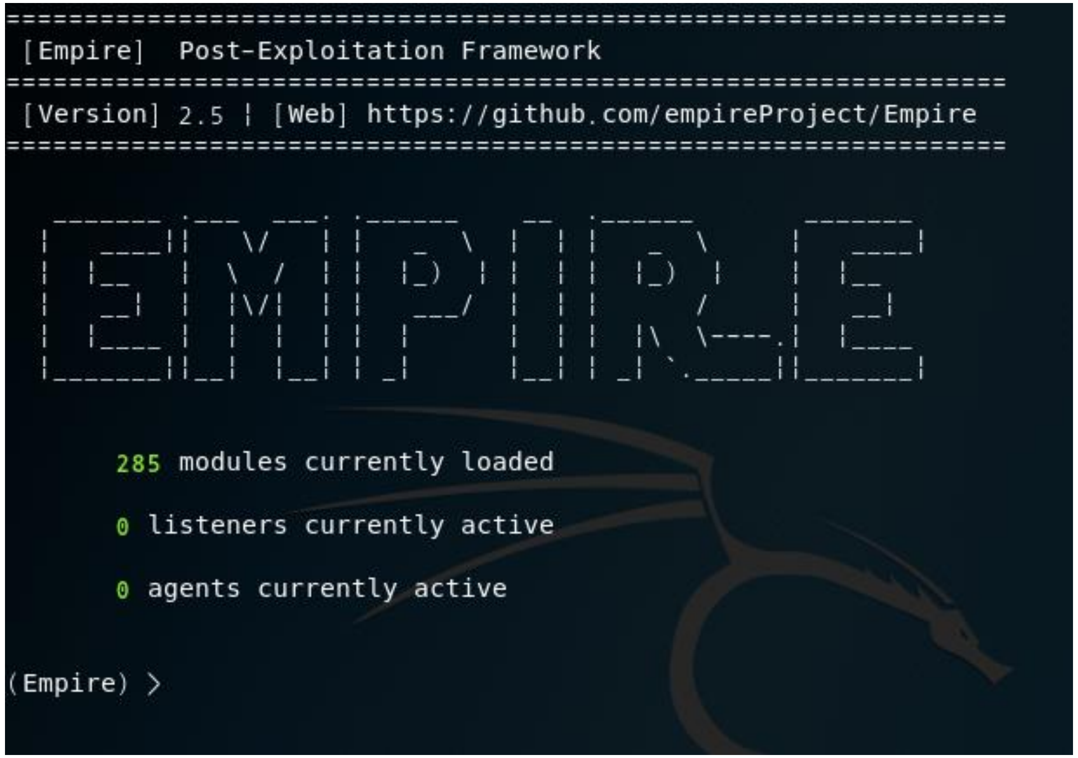
SSDT Hooking

선택 관리자: 명령 프롬프트

```
Entry 0x1022: 0xfffff801cffad110 (NtRequestWaitReplyPort) owned by ntoskrnl.exe
Entry 0x1023: 0xfffff801cfee8f9c (NtQueryVirtualMemory) owned by ntoskrnl.exe
Entry 0x1024: 0xfffff801cfebf590 (NtOpenThreadToken) owned by ntoskrnl.exe
Entry 0x1025: 0xfffff801cfebc390 (NtQueryInformationThread) owned by ntoskrnl.exe
Entry 0x1026: 0xfffff801cfeeecb0 (NtOpenProcess) owned by ntoskrnl.exe
Entry 0x1027: 0xfffff801cfad96c0 (NtSetInformationFile) owned by ntoskrnl.exe
Entry 0x1028: 0xfffff801cfeeaee0 (NtMapViewOfSection) owned by ntoskrnl.exe
Entry 0x1029: 0xfffff801cff668a8 (NtAccessCheckAndAuditAlarm) owned by ntoskrnl.exe
Entry 0x102a: 0xfffff801cfeb5164 (NtUnmapViewOfSection) owned by ntoskrnl.exe
Entry 0x102b: 0xfffff801cfeb6210 (NtReplyWaitReceivePortEx) owned by ntoskrnl.exe
Entry 0x102c: 0xfffff801cfbc1ca0 (NtTerminateProcess) owned by ntoskrnl.exe
Entry 0x102d: 0xfffff801d01d8c3c (NtSetEventBoostPriority) owned by ntoskrnl.exe
Entry 0x102e: 0xfffff801cffcc238 (NtReadFileScatter) owned by ntoskrnl.exe
Entry 0x102f: 0xfffff801cfebf5b0 (NtOpenThreadTokenEx) owned by ntoskrnl.exe
Entry 0x1030: 0xfffff801cfeeeeb0 (NtOpenProcessTokenEx) owned by ntoskrnl.exe
Entry 0x1031: 0xfffff801cffa3a40 (NtQueryPerformanceCounter) owned by ntoskrnl.exe
Entry 0x1032: 0xfffff801cfed1b20 (NtEnumerateKey) owned by ntoskrnl.exe
Entry 0x1033: 0xfffff801cff7eb50 (NtOpenFile) owned by ntoskrnl.exe
Entry 0x1034: 0xfffff801cfef3ef0 (NtDelayExecution) owned by ntoskrnl.exe
Entry 0x1035: 0xfffff801cffb85b0 (NtQueryDirectoryFile) owned by ntoskrnl.exe
Entry 0x1036: 0xfffff801cfef04b0 (NtQuerySystemInformation) owned by ntoskrnl.exe
```

```
0x9000 ₩??₩C:₩W
0xf000 ₩??₩C:₩P
8-45B4-8919-12C2
0x8000 ₩??₩C:₩U
```

Powershell



[Empire 실행 화면]

- 공격자 서버와 연결
- Credential 획득
- Dropper
- Key Logging

04

VOLATILITY FOUNDATION

About

The Volatility Foundation is an independent 501(c) (3) non-profit organization that maintains and promotes open source memory forensics with The Volatility Framework.

Releases

The Volatility Framework is open source and written in Python. Releases are available in zip and tar archives, Python module installers, and standalone executables.

OMFW

The Open Memory Forensics Workshop (OMFW) is a half-day event where participants learn about innovative, cutting-edge research from the industry's leading analysts.

Contest

The Volatility Plugin Contest is your chance to win cash, shwag, and the admiration of your peers while giving back to the community. Warning: competition may be fierce!

04

VOLATILITY FOUNDATION

About

The Volatility Foundation is an independent 501(c) (3) non-profit organization that maintains and [illegible] with The Volatility Framework.

Releases

The Volatility Framework is open source and written in Python. Releases are available in zip and tar archives, Pytho[illegible] executables.

OMFW

The Open Memory Forensics Workshop (OMFW) is a half-day event where participants learn about innovative, cutting-edge research from the industry's leading analysts.

Contest

The Volatility Plugin Contest is your chance to win cash, shwag, and the admiration of your peers while giving back to the community. Warning: competition may be fierce!

2018 Volatility Analysis Contest

악성코드 샘플과 시나리오를 선택하고

Volatility를 사용하여 분석 보고서 작성

04

"Olympic Destroyer: who hacked the Olympics?"

The PyeongChang Winter Olympic Games started with a scandal: unknown hackers attacked the servers just before the opening ceremonies and many spectators were unable to attend the ceremonies as they were unable to print out tickets.

[관련 기사_Kaspersky_180309]

[당시 평창 조직위 홈페이지 오류 화면]

2018 Volatility Analysis Contest

In order to analyze the provided memory sample, I used the "imageinfo" pluggin to identify the operating system information as shown below.

```
C:\WINDOWS\system32\cmd.exe

C:\volatility-master>vol.py -f C:\Olympic\4\Windows7-1a1299dc.vmem imageinfo
Volatility Foundation Volatility Framework 2.6
INFO    : volatility.debug    : Determining profile based on KDBG search...
          Suggested Profile(s) : Win7SP1x86_23418, Win7SP0x86, Win7SP1x86_24000, Win7SP1x86
                     AS Layer1 : IA32PagedMemoryPae (Kernel AS)
                     AS Layer2 : FileAddressSpace (C:\Olympic\4\Windows7-1a1299dc.vmem)
                      PAE type : PAE
                           DTB : 0x185000L
                          KDBG : 0x82f7ac68L
          Number of Processors : 2
     Image Type (Service Pack) : 1
                KPCR for CPU 0 : 0x82f7bd00L
                KPCR for CPU 1 : 0x807d5000L
             KUSER_SHARED_DATA : 0xffdf0000L
           Image date and time : 2018-09-23 16:28:51 UTC+0000
     Image local date and time : 2018-09-24 01:28:51 +0900
```

Figure 1 – imageinfo

As a result of identification, victim's PC was running a Windows 7 32bit operating system, and since there are two fields related to KPCR, victim's PC has two central processing units.

After identifying the operating system according to NIST 800-86 and IETF RFC 3227, I conducted process analysis, result of checking the start time of process's are divided into four major parts as shown below.

04

Volatility Analysis Contest

대회 준비를 통한 메모리 분석 능력 향상

"Olympic Destroyer: who hacked the Olympics?"

The PyeongChang Winter Olympic Games started with a scandal: unknown hackers attacked the servers just before the opening ceremonies and many spectators were unable to attend the ceremonies as they were unable to print out tickets.

[관련 기사_Kaspersky_180309]

[당시 평창 조직위 홈페이지 오류 화면]

2018 Volatility Analysis Contest

In order to analyze the provided memory sample, I used the "imageinfo" pluggin to identify the operating system information as shown below.

```
C:\WINDOWS\system32\cmd.exe
C:\volatility-master>vol.py -f C:\Olympic\4\Windows7-1a1299dc.vmem imageinfo
Volatility Foundation Volatility Framework 2.6
INFO    : volatility.debug    : Determining profile based on KDBG search...
                      : Win7SP1x86_23418, Win7SP0x86, Win7SP1x86_24000, Win7SP1x86
                      : IA32PagedMemoryPae (Kernel AS)
           AS Layer2 : FileAddressSpace (C:\Olympic\4\Windows7-1a1299dc.vmem)
            PAE type : PAE
                 DTB : 0x185000L
     KPCR for CPU 0 : 0x82f7bd00L
     KPCR for CPU 1 : 0x807d5000L
   KUSER_SHARED_DATA : 0xffdf0000L
 Image date and time : 2018-09-23 16:28:51 UTC+0000
Image local date and time : 2018-09-24 01:28:51 +0900
```

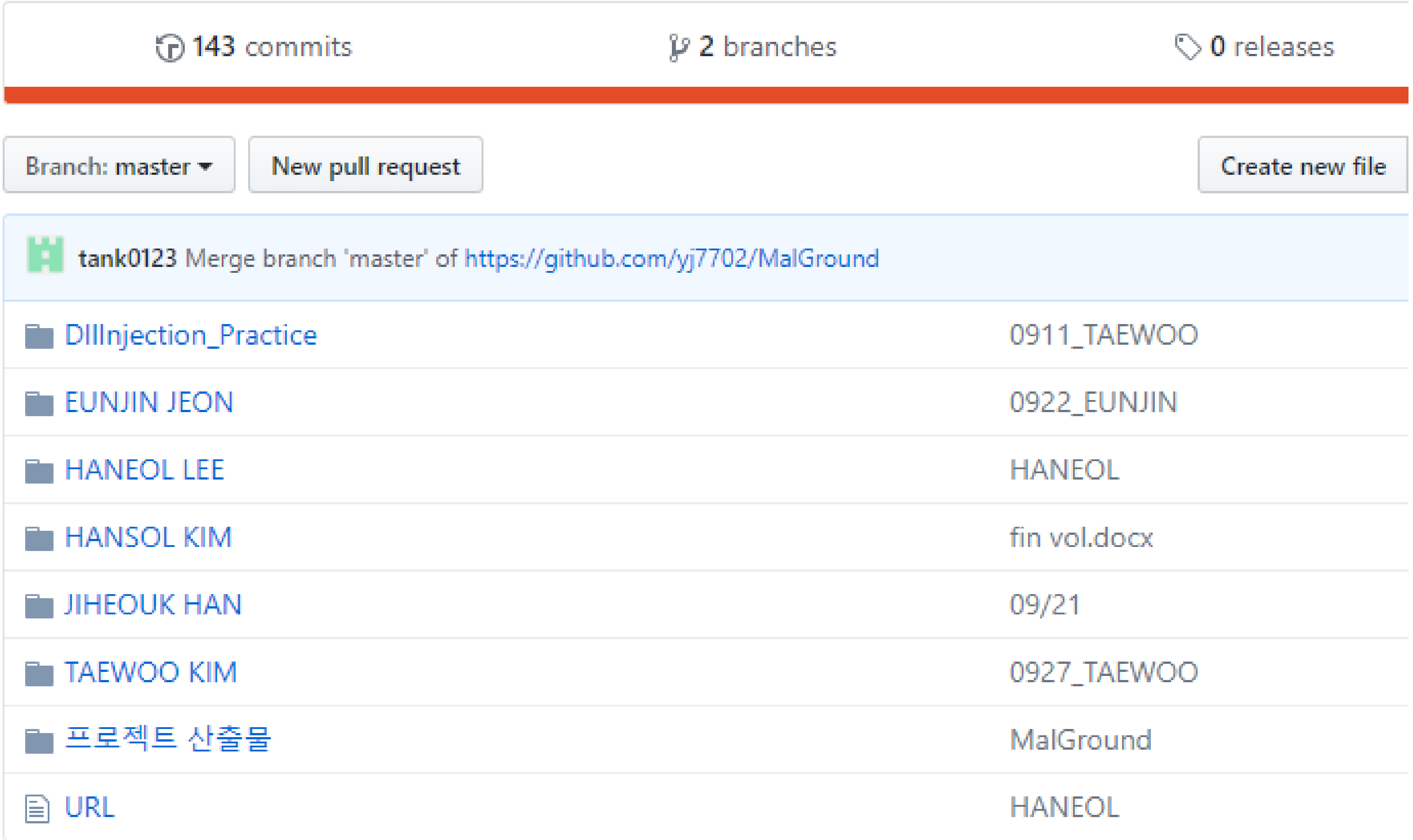
Figure 1 – imageinfo

As a result of identification, victim's PC was running a Windows 7 32bit operating system, and since there are two fields related to KPCR, victim's PC has two central processing units.

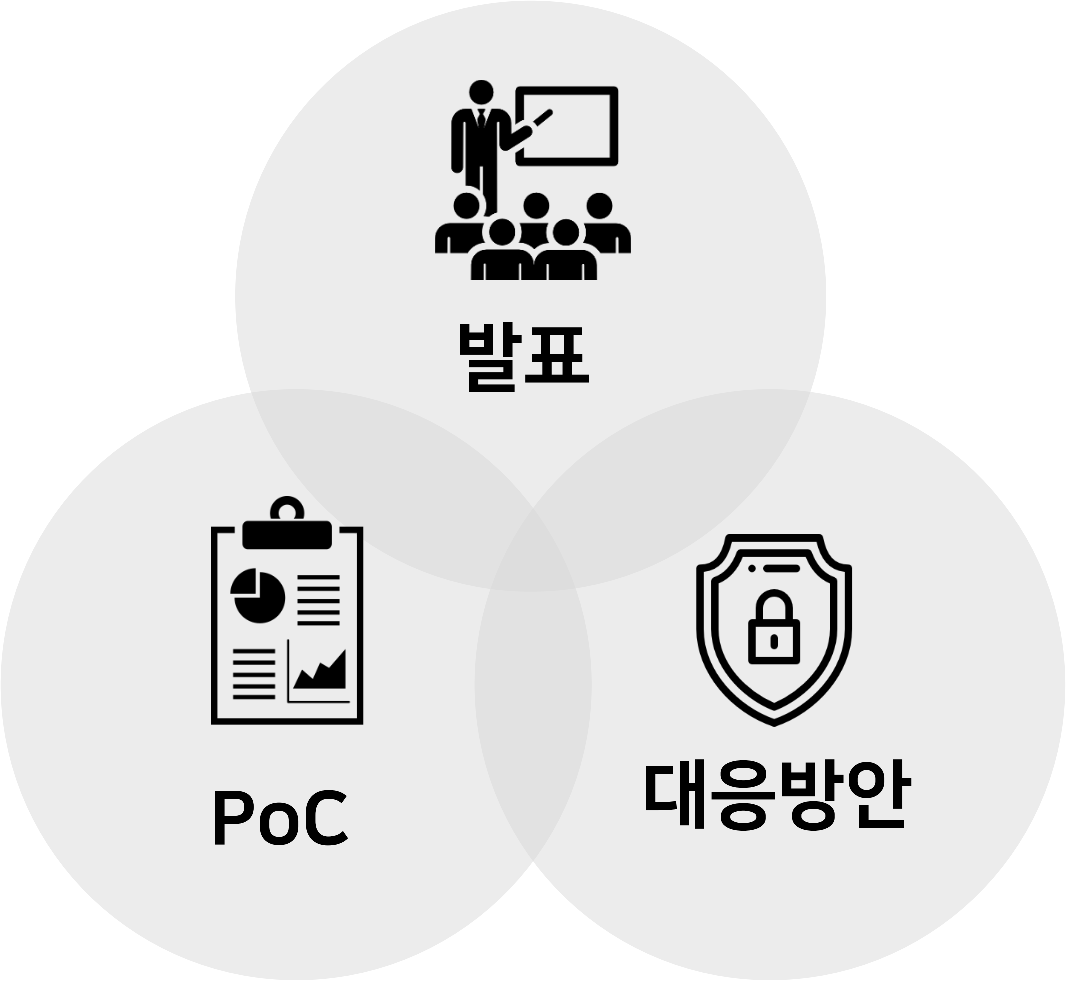
After identifying the operating system according to NIST 800-86 and IETF RFC 3227, I conducted process analysis, result of checking the start time of process's are divided into four major parts as shown below.

04

Github

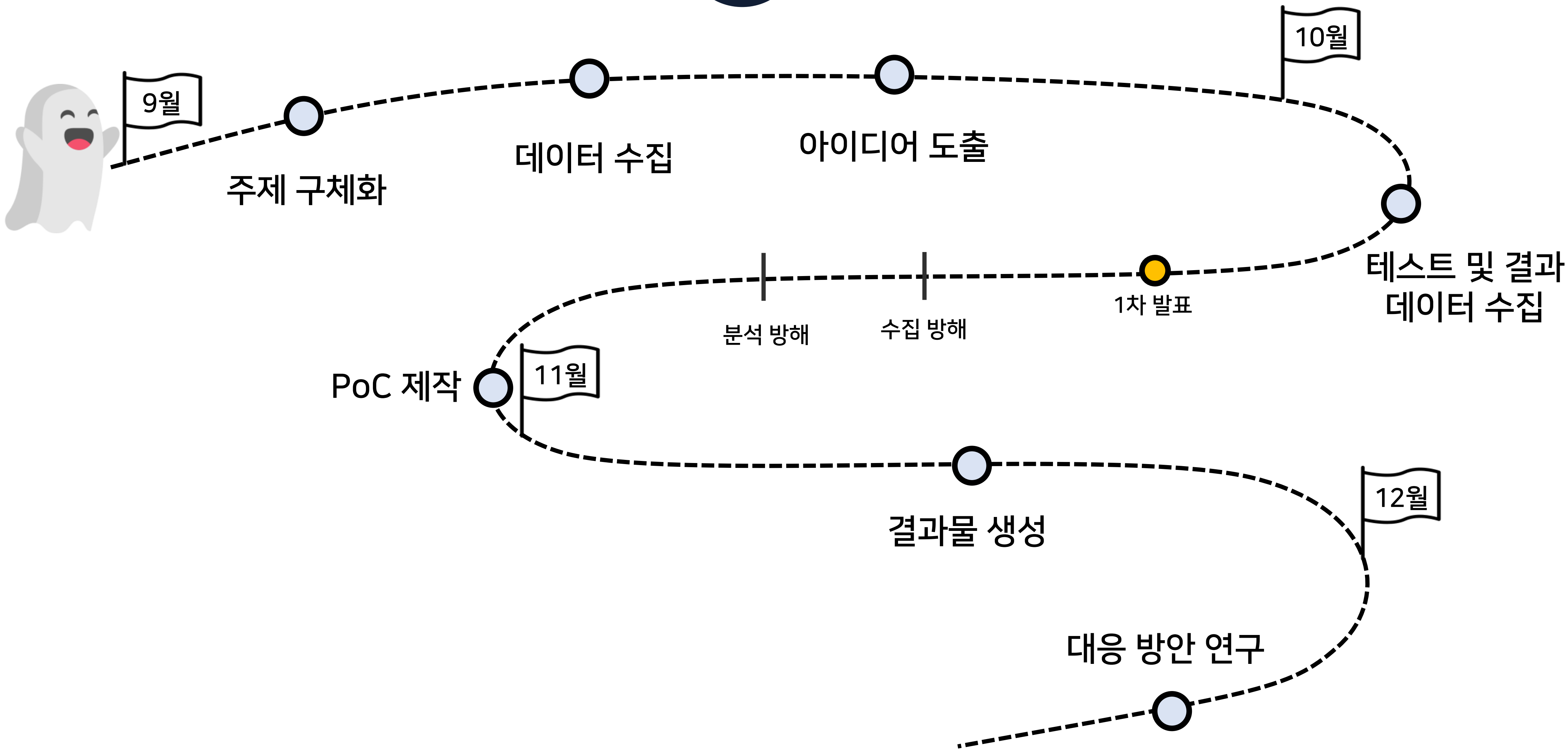

143 commits
2 branches
0 releases
Branch: master
New pull request
Create new file
tank0123 Merge branch 'master' of https://github.com/yj7702/MalGround
DllInjection_Practice
0911_TAEWOO
EUNJIN JEON
0922_EUNJIN
HANEOL LEE
HANEOL
HANSOL KIM
fin vol.docx
JIHEOUK HAN
09/21
TAEWOO KIM
0927_TAEWOO
프로젝트 산출물
MalGround
URL
HANEOL

05



6. 일정

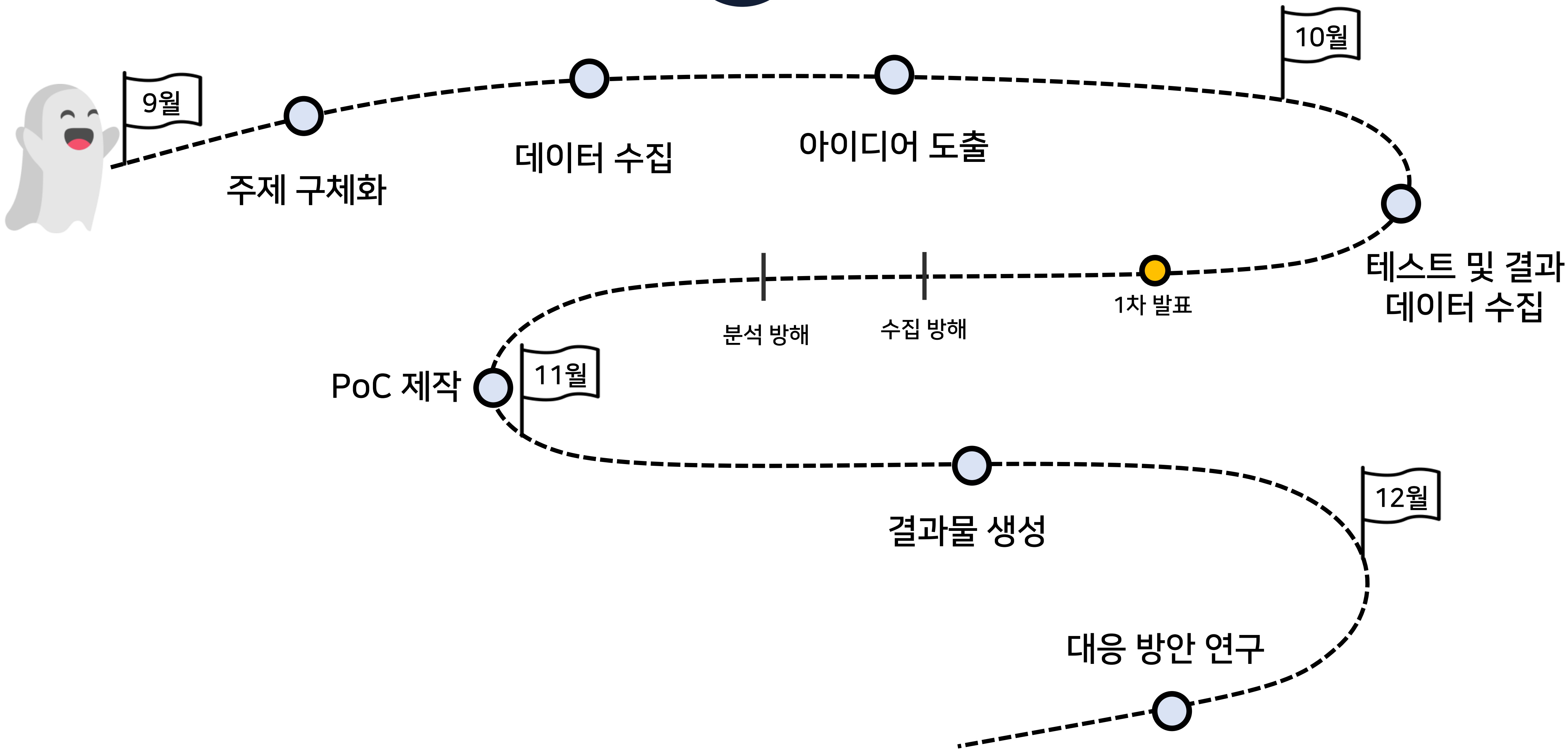
06



9월
주제 구체화
데이터 수집
아이디어 도출
10월
테스트 및 결과
데이터 수집
1차 발표
수집 방해
분석 방해
PoC 제작
11월
결과물 생성
12월
대응 방안 연구

6. 일정

06


9월
주제 구체화
데이터 수집
아이디어 도출
10월
테스트 및 결과
데이터 수집
1차 발표
수집 방해
분석 방해
PoC 제작
11월
결과물 생성
12월
대응 방안 연구

감사합니다